

React Performance Optimization Guide

React re-renders components when state or props change. Unnecessary re-renders degrade performance, especially in large component trees. This guide covers proven optimization techniques.

React.memo for Pure Components

React.memo is a higher-order component that memoizes the rendered output. It performs a shallow comparison of props and skips re-rendering if props haven't changed. Use it for components that render often with the same props. Avoid wrapping components that receive new object/array props on every render, as the shallow comparison will always detect changes.

useMemo and useCallback Hooks

useMemo caches the result of expensive computations. useCallback caches function references to prevent child components from re-rendering when callback props haven't logically changed. Both accept a dependency array that controls when the cached value is recomputed. Overusing these hooks adds complexity without benefit; profile first, optimize second.

Virtualization for Large Lists

Rendering thousands of DOM nodes causes jank. Libraries like react-window and react-virtuoso render only visible items, dramatically reducing DOM nodes. For a list of 10,000 items at 50px height in a 500px viewport, only ~10 items are in the DOM at any time instead of 10,000.

Code Splitting with React.lazy

React.lazy enables dynamic imports, splitting bundles into smaller chunks loaded on demand. Combined with Suspense for loading states, this reduces initial bundle size. Route-based splitting is the most common pattern, loading page components only when the user navigates to them.